



第9课时: for 循环

语法结构 · while 对比 · 变量作用域 · 典型应用



为什么需要 for 循环？



while 循环适合“条件驱动”的场景，但很多循环具有明显的计数特征：

- 遍历数组/容器（索引从 0 到 N-1）
- 执行固定次数的重复操作（如打印 N 次）
- 需要集中管理循环变量的初始化、条件和更新

for 循环将这三要素整合在一行，代码更紧凑、可读性更强，是 C++ 中最常用的循环结构。



for 循环的语法



```
for (初始化; 条件表达式; 更新表达式) {  
    // 循环体  
}
```

各部分说明：

部分	作用	示例
初始化	声明或赋值循环变量（仅执行一次）	<code>int i = 0;</code>
条件	每次迭代前检查，为真则继续	<code>i < n;</code>
更新	每次循环体执行后执行	<code>i++</code>

执行流程



1. **初始化** 执行一次（通常定义循环变量）。
2. **检查条件**，若为 `true` 则执行循环体，否则跳出。
3. 循环体执行完毕后，执行 **更新** 表达式。
4. 回到步骤 2，重复直到条件为 `false`。

“ 等价于 `while` 版本：

```
初始化;  
while (条件) {  
    循环体;  
    更新;  
}
```

示例：打印 1 到 5



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        cout << i << " ";
    }
    return 0;
}
// 输出: 1 2 3 4 5
```

- 初始化: `int i = 1`
- 条件: `i <= 5`
- 更新: `i++`
- 循环体: `cout << i << " ";`

for 与 while 的对比与转换



特点	for	while
适用场景	已知循环次数（计数循环）	条件驱动，次数未知
代码紧凑性	三要素集中，一目了然	三要素分散，可能遗忘更新
变量作用域	初始化部分声明的变量仅限循环内	变量作用域取决于声明位置

相互转换示例

for → while:

```
for (int i=0; i<10; i++) { cout << i; }// 等价于
int i = 0;
while (i < 10) {
    cout << i;
    i++;}
```

while → for（如果 while 有明确的初始化、条件和更新）也可以改写。



循环变量的作用域



在 `for` 初始化部分声明的变量，其作用域**仅限于 for 循环体内**，循环结束后即销毁。

```
for (int i = 0; i < 5; i++) {  
    cout << i;    // 合法  
}  
// cout << i;    // 错误: i 未定义 (超出作用域)
```

若需要在循环外使用循环变量的最终值，必须在循环**之前**声明变量：

```
int i;  
for (i = 0; i < 5; i++) { /* ... */ }  
cout << i;    // 输出 5, 合法
```



错误类型	示例	解决方法
分号误用	<code>for (int i=0; i<10; i++);</code> 循环体为空	去掉分号或使用大括号
条件边界	<code>i < n</code> VS <code>i <= n</code> 差一次	根据需求确定
变量作用域	循环外使用循环内声明的变量	提前声明变量
更新遗漏	将更新写在循环体内但忘记递增	使用标准的更新表达式



例题1：求 1 到 N 的和（for 实现）



题目：输入 n ，计算 $1+2+\dots+n$ 。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    cout << sum;
    return 0;
}
```

例题2：求 N 的阶乘（for 实现）



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    cout << fact;
    return 0;
}
```

“ 若 $n=0$ ，循环不执行，fact 保持 1，正确。 ”

例题3：输出偶数（for 步长控制）



题目：输入 n ，输出 $1 \sim n$ 中的所有偶数，空格隔开。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 2; i <= n; i += 2) {
        cout << i << " ";
    }
    return 0;
}
```

例题4：反向输出数字（for 也能实现）



虽然 `while` 更适合，但 `for` 也可以：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (; n > 0; n /= 10) { // 省略初始化
        cout << n % 10 << " ";
    }
    return 0;
}
```

“ 注意：省略初始化部分，但分号必须保留。 ”

例题5：统计数字位数（for 实现）



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    int count = 0;
    for (; n > 0; n /= 10) {
        count++;
    }
    cout << count;
    return 0;
}
```

例题6：嵌套 for 循环——打印矩形



题目：输入 n, m ，打印 n 行 m 列的 `*` 矩阵。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            cout << "*";
        }
        cout << endl;    // 每行换行
    }
    return 0;
}
```

“ 外层控制行，内层控制列。”

例题7：打印乘法口诀表（完整版）



输入 n，打印下三角乘法表：

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << i << "*" << j << "=" << i*j << "\t";
        }
        cout << endl;
    }
    return 0;
}
```

例题8：斐波那契数列（for 实现）



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    if (n == 1 || n == 2) {
        cout << 1;
        return 0;
    }
    int a = 1, b = 1;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    cout << b;
    return 0;
}
```

例题9：判断素数



题目：输入正整数 n ，判断它是否为素数（质数）。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    if (n < 2) {
        cout << "No";
        return 0;
    }
    bool isPrime = true;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            isPrime = false;
            break;
        }
    }
    cout << (isPrime ? "Yes" : "No");
    return 0;
}
```

例题10：求最大值



题目：输入 n 个整数，输出其中的最大值。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, x, maxVal;
    cin >> n;
    cin >> maxVal;    // 先读第一个数作为初始最大值
    for (int i = 2; i <= n; i++) {
        cin >> x;
        if (x > maxVal) maxVal = x;
    }
    cout << maxVal;
    return 0;
}
```

for 循环的变体



1. 无限循环

```
for (;;) {    // 相当于 while(true)
    // 需用 break 退出
}
```

2. 省略部分表达式

- 省略初始化：变量已在外部定义
- 省略条件：默认为 true（无限循环）
- 省略更新：可在循环体内更新

```
int i = 0;
for (; i < 10; ) {
    cout << i;
    i++;    // 更新在体内
}
```

总结



- **for** 循环将初始化、条件、更新集中管理，适合计数循环。
- 与 **while** 可以互相转换，选择取决于场景和可读性。
- 循环变量的**作用域**在初始化部分声明的变量仅限循环内。
- 掌握 **for** 是编写高效、清晰 C++ 代码的基础。

下节课将学习 **do-while** 循环，它至少执行一次循环体。



谢谢大家！



循环让程序拥有强大的重复执行能力！

Q & A

